

Write C program to simulate:

a) Banker's algorithm for purpose of deadlock avoidance.

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define MAX 10
```

```
int main()
```

```
{
    int n, m; // n = processes, m = resources;

```

```
    int allocated[MAX][MAX], max[MAX][MAX], need[MAX][MAX];

```

```
    int available[MAX];

```

```
    int work[MAX];

```

```
    bool finish[MAX];

```

```
    int safeSequence[MAX];

```

```
    printf("Enter no. of processes: ");

```

```
    scanf("%d", &n);

```

```
    printf("Enter no. of resources: ");

```

```
    scanf("%d", &m);

```

```
    printf("Enter Allocation Matrix: \n");

```

```
    for (int i = 0; i < n; i++) {

```

```
        for (int j = 0; j < m; j++) {

```

```
            scanf("%d", &allocated[i][j]);

```

```
        }

```

```
    }

```

```
    printf("Enter Max Demand Matrix: \n");

```

```
    for (int i = 0; i < n; i++) {

```

```
        for (int j = 0; j < m; j++) {

```

```
            scanf("%d", &max[i][j]);

```

```
        }

```

```
    }

```

```
    printf("Enter Available Resources: \n");

```

```
    for (int i = 0; i < m; i++) {

```

```
        for (int j = 0; j < n; j++) {

```

```
            scanf("%d", &max[i][j]);

```

```
        }

```

```
    }

```



```
printf ("Enter Available Resources: \n");
```

```
for (int i=0; i<m; i++) {
```

```
scanf ("%d", &available[i]);
```

```
}
```

```
for (int i=0; i<n; i++) {
```

```
for (int j=0; j<m; j++) {
```

```
need[i][j] = max(need[i][j], allocated[i][j]);
```

```
}
```

```
for (int i=0; i<m; i++) {
```

```
work[i] = available[i];
```

```
}
```

```
for (int i=0; i<n; i++) {
```

```
finish[i] = false;
```

```
}
```

```
int count=0;
```

```
while (count<n) {
```

```
bool found = false;
```

```
for (int i=0; i<n; i++) {
```

```
if (finish[i] == false) {
```

```
bool canExecute = true;
```

```
for (int j=0; j<m; j++) {
```

```
if (need[i][j] > work[j]) {
```

```
canExecute = false;
```

```
break;
```

```
}
```

```
if (canExecute) {
```

```
for (int j=0; j<m; j++) {
```

```
work[j] += allocated[i][j];
```

```
}
```

```
safeSequence[count++] = i;
```



```
finish [i] = true;
```

```
found = true;
```

```
{
```

```
}
```

```
}
```

```
if (!found) {
```

```
break;
```

```
}
```

```
}
```

```
if (count == n) {
```

```
printf ("In System is in a safe state.\n");
```

```
printf ("Safe sequence is : ");
```

```
for (int i = 0; i < n; i++) {
```

```
printf ("P%d", safeSequence[i]);
```

```
if (i != n - 1)
```

```
printf (" ");
```

```
}
```

```
printf ("\n");
```

```
} else {
```

```
printf ("In System is in an UNSafe state (Deadlock may occur).\n");
```

```
}
```

```
return 0;
```

```
printf ("Enter Process number: ");
```

```
scanf ("%d", &p);
```

```
printf ("Enter Request \n");
```

```
for (int i = 0; i < m; i++)
```

```
scanf ("%d", &request[i]);
```

```
for (int i = 0; i < m; i++) {
```

```
if (request[i] > need[p][i]) {
```

```
printf ("Exceeds need\n");
```

```
return 0;
```

```
}
```

```
if (request[p] > avail[i]) {
```

```
printf ("Not available\n");
```

```
}
```

```
}
```

```
for (int i = 0; i < m; i++) {
```

```
avail[i] -= request[i];
```

```
alloc[p][i] += request[i];
```

```
need[p][i] -= request[i];
```

```
}
```

```
return 0;
```

```
}
```


Output

Enter no. of processes: 5

Enter no. of resources: 4

Enter Allocation Matrix:

0 0 1 2

1 0 0 0

1 3 5 4

0 6 3 2

0 0 1 4

Enter Maximum Demand Matrix:

0 0 1 2

1 7 5 0

2 3 5 6

0 6 5 2

0 6 5 6

Enter Available Resources:

1 5 2 0

System is in a safe state.

Safe sequence is: $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

Enter process number: 1

Enter request:

0 2 1 0

Request Granted

Single
request

P	Allocation	MAX	Available	Need
	A B C D	A B C D	A B C D	A B C D
P ₀	0 0 1 2	0 0 1 2	1 5 2 0	0 0 0 0
P ₁	0 0 0 0	1 7 5 6		0 7 5 0
P ₂	1 3 5 4	2 3 5 6		1 0 0 2
P ₃	0 6 3 2	0 6 5 2		0 0 2 0
P ₄	0 0 1 4	0 6 5 6		0 6 0 4
Total:	2 9 10 12			
	1 5 2 0			

3 14 12 12

P₀ Finish[0] = falseP₄ Finish[4] = false

Need ≤ available

0 0 0 0 ≤ 1 5 2 0

0 0 0 0 ≤ 1 5 2 0

(work) work = work + Allocation

work = work + Allocation

(work) work = 2 14 11 8 + 0 0 1 4

= 1 5 2 0 + 0 0 1 2

(work) work = 2 14 11 8 + 0 0 1 2

= 1 5 3 2

Finish[4] = true

Finish[0] = true

P₁ Finish[1] = falseP₂ Finish[2] = false

Need ≤ available

(work) work = 2 14 11 8 + 0 7 5 0 ≤ 2 14 12 12

0 7 5 0 ≤ 1 5 3 2

(work) work = work + Allocation

= 2 14 12 12 + 1 0 0 0

= 3 14 12 12

P₂ Finish[2] = false

1 0 0 2 ≤ 1 5 3 2

(work) work = 3 14 12 12 + 1 0 0 2

work = work + Allocation

(work) work = 3 14 12 12 + 1 0 0 2

= 1 5 3 2 + 1 3 5 4

(work) work = 3 14 12 12 + 1 0 0 2

= 2 8 8 6

(work) work = 3 14 12 12 + 1 0 0 2

Finish[2] = true

P₃ Finish[3] = false

0 0 2 0 ≤ 2 8 8 6

(work) work = 3 14 12 12 + 1 0 0 2

work = work + Allocation

(work) work = 3 14 12 12 + 1 0 0 2

= 2 8 8 6 + 0 6 3 2

= 2 14 11 8